Federated Intrusion Detection Under Label-Flipping Poisoning on CICIoT2023

Salvador Leon Lopez, Elena Victoria Sanchez, Joe Reyna, and April Flores

***Abstract*—We simulate a ten-client federated intrusion detector on CICIoT2023 (all 63 files, 306,124 flows; eight paired conditions, three seeds each) and measure what label-flipping clients cost and whether robust aggregation recovers it. One attacker relabeling half of its attack rows as benign lowers FedAvg's macro F1 by only 1.1 points, so our 3-point hypothesis is not supported. The damage is concentrated: missed attacks rise from 7.2% to 10.6%, almost all in the four families that resemble benign traffic. With one attacker neither the unweighted mean nor trimmed mean helps; with two or three, both do, through dropping size weighting rather than trimming. Results hold at 60 rounds and without the IAT feature.**

***Index Terms*—Federated learning, intrusion detection, data poisoning, label flipping, robust aggregation, CICIoT2023.**

I. Research Question and Changes

ORGANIZATIONS that see different attack traffic could train a shared intrusion detector with federated learning (FL) [1] without pooling raw captures, but the server cannot inspect client data, so a compromised client can poison the model [2]. *RQ:* in a ten-client federated IDS on non-IID partitions of CICIoT2023 [3], how much detection performance is lost when one client flips labels, and does coordinate-wise trimmed mean [4] recover it? *Hypotheses:* (H1) FedAvg macro F1 falls by more than 3 points versus the paired clean run; (H2) trimmed mean recovers at least half of that loss.

Changes after Milestone 1 feedback: a documented grouped split with per-family supports (Sec. II-B); the 46→39 feature mapping (Table II); an *unweighted-mean* control; definitions of label-informed held-out assignment and PR-AUC (Sec. III-A); and, as requested, the paired comparison and per-family/per-client error analysis before the attacker sweep, whose 3-attacker case is a stress test.

II. Data and Experimental Design

A. Subset

We read all 63 MERGED_CSV files (45,019,234 rows; 9 rows with an empty label dropped). From each file we sample at most 1,429 rows per family (575,321 rows), then remove 19 inf/NaN rows, 27,282 exact (feature+label) duplicates and 14,033 rows whose features occur with conflicting labels, *before* splitting. Each family is capped at 45,000 rows; Web (23,615) and BruteForce (12,509) are kept whole. The subset has 306,124 rows; its family prevalence is a design choice (raw DDoS is 72.3%, subset 14.7%).

The authors are Group 8 in CSCI 6330: Foundations of Intelligent Security Systems, Department of Computer Science, The University of Texas Rio Grande Valley.

Milestone 2 progress report, November 2026. Code: github.com/Jreyna4/CSCI-6330-Foundation-Security-Systems-Course-Project.

B. Grouped split

Each source file goes entirely into one split, so fractions move in steps of about $1/63 \approx 1.6\%$ of the rows (the Milestone 1 pilot's 12 files gave 8.3% steps, hence 74.4/8.5/17.0%). Of 2,000 seeded file assignments we keep the one closest to 70/10/20 and to the overall family mix (Table I); the file list and per-file counts are in `split_files.csv`.

Limitation. Every file contains all eight families, so one capture can span several files, and the release has no capture id, IP or timestamp to group on. A nearest-neighbour check confirms this: test rows are as close to their nearest training row (median 0.120, standardized features) as training rows are to each other (0.131). Results measure generalization within one testbed, not to unseen captures.

C. Features

The documentation lists 46 flow features; our release has 39 (Table II), checked on every run (`feature_schema.csv`). We do no feature selection: none of the 39 is an identifier (no IPs, ports, MACs or timestamps), the 8 missing ones are not in the files, and selecting on our own data would add a leakage-prone tuning step. Each feature gets $\operatorname{sign}(x)\log(1+|x|)$ (heavy-tailed counts and rates), then standardization fit on the training pool. Dropping IAT, a possible timing shortcut, leaves centralized macro F1 unchanged (.745 vs. .741; Sec. IV-A).

D. Clients and attacker

Ten clients are formed by Dirichlet label skew ($\alpha = 0.5$) [5]. Automatic checks (share $\geq$2%, size ratio $\leq$20, purity $\leq$95%, $\geq$2 families with $\geq$50 rows) pass for Dirichlet (11,262–31,560 rows; max purity 0.83) and IID, and fail for k-means again (175–49,318 rows; one client 99.9% one family), so Dirichlet stays primary, as in Milestone 1. The attacker is fixed: the client with the most attack rows, here client 5 (31,560 rows, 14.7% of the pool, mostly Spoofing and DDoS). Per seed it relabels a fixed 50% of each of its attack families as Benign (about 15,460 rows, 7.2% of the pool) and otherwise trains honestly.

Held-out rows are assigned to clients with the frozen per-family Dirichlet proportions, i.e. using their *labels*: label-informed benchmark construction, not a client-identification rule, acceptable because labels are never model inputs and the test set is never used for tuning.

III. Methodology

Model and training (unchanged, identical in every condition): MLP 39–128–64–8 (ReLU, dropout 0.1; 13,896 parameters), Adam (lr 10^{-3}), batch 256, cross-entropy; 30 FL rounds

TABLE I
GROUPED SPLIT (WHOLE FILES) AND PER-FAMILY SUPPORTS

Split	Files	Rows	Share	Benign	DDoS	DoS	Recon	Web	BruteForce	Spoofing	Mirai
Train	44	214,725	70.1%	31,434	31,443	31,335	31,648	16,791	8,941	31,730	31,403
Val	6	30,045	9.8%	4,375	4,505	4,525	4,253	2,468	1,301	4,312	4,306
Test	13	61,354	20.0%	9,191	9,052	9,140	9,099	4,356	2,267	8,958	9,291

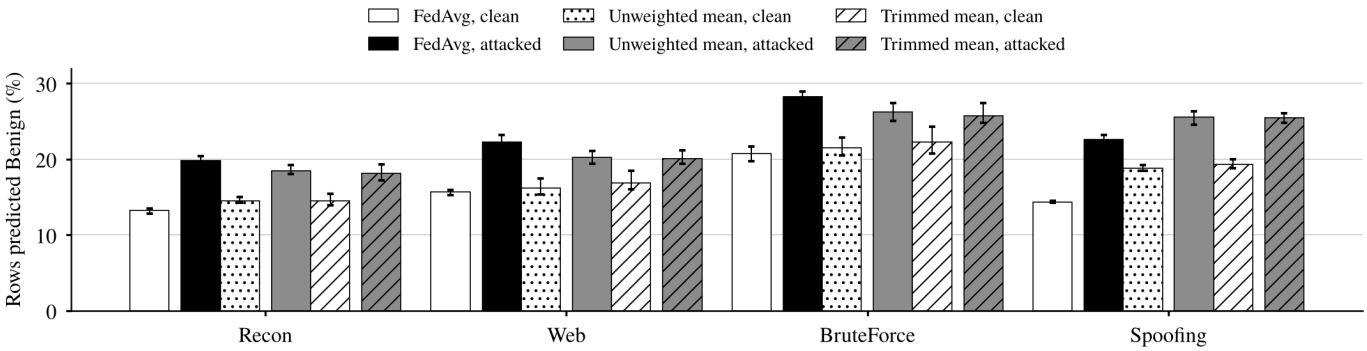


Fig. 1. Share of each family's validation rows predicted Benign, clean vs. attacked, for each aggregation rule (mean over 3 seeds; bars show the range). DDoS, DoS and Mirai are never predicted Benign (0.0% in all conditions) and are omitted.

TABLE II
OFFICIAL 46 FEATURES → 39 MODEL INPUTS

Status	Features
Kept (37)	Header_Length, Protocol Type, Rate, 7 flag numbers, ack/syn/fin/rst counts, HTTP, HTTPS, DNS, Telnet, SMTP, SSH, IRC, TCP, UDP, DHCP, ARP, ICMP, IPv, LLC, Tot sum, Min, Max, AVG, Std, Tot size, IAT, Number, Variance
Renamed (1)	Duration (TTL) → Time_To_Live
Added (1)	IGMP
Not in release (8)	flow_duration, Srate, Drate, urg_count, Magnitue, Radius, Covariance, Weight

TABLE III
VALIDATION RESULTS, MEAN ± STD OVER 3 SEEDS

Condition	Macro F1	FNR	FPR	Worst client F1
Centralized	.741 ± .002	.068	.205	–
FedAvg, IID	.709 ± .001	.075	.223	.692
FedAvg, non-IID	.691 ± .003	.072	.253	.471
Unweighted mean	.690 ± .002	.082	.233	.497
Trimmed mean	.687 ± .000	.084	.229	.487
FedAvg, attacked	.680 ± .002	.106	.174	.477
Unweighted mean, attacked	.682 ± .001	.106	.182	.485
Trimmed mean, attacked	.676 ± .002	.105	.183	.460

of one local epoch with all ten clients in our own simulation loop (Flower [6] deprecated its simulation API). Each round moves 1.11 MB (33.4 MB per run); a run takes about 33 s on a MacBook Pro CPU (Python 3.14, PyTorch 2.14).

Aggregation. FedAvg averages client models weighted by client size. The *unweighted mean* gives every client equal weight. Trimmed mean ($\beta = 0.2$) drops, per coordinate, the 2 highest and 2 lowest of the 10 values and averages the remaining 6 without weighting. FedAvg→mean isolates the effect of size weighting; mean→trimmed isolates trimming. With one attacker its values *can* all be trimmed, but label flipping need not make them the extreme ones; with three attackers they can survive in every coordinate, so that case is only a stress test.

Design and pairing. Eight conditions (Table III): centralized; FedAvg on IID and Dirichlet clients; and FedAvg, unweighted mean and trimmed mean on Dirichlet clients, clean and attacked. The data seed fixes subset, split and partition; run seeds 42–44 change initialization, batch order and poisoned rows, and same-seed runs start from identical weights, so effects are paired differences (Table IV), reported as mean, std and range. All results are on validation data; the test set is used once, at the end.

A. Metrics

Macro F1 averages per-family F1 over families with at least one true row. *Per-family PR-AUC* is one-vs-rest average precision with that family's softmax probability as the score; *macro PR-AUC* averages it over families with at least one positive and one negative row. Families with no support are excluded and listed, never scored 0 or 1, which matters for small per-client held-out sets. We also report the binary miss rate (FNR) and false-alarm rate (FPR), per-family share of rows predicted Benign (the attacker's target), and per-client metrics on each client's held-out rows.

IV. PRELIMINARY RESULTS

Global effect. Federated training costs 3.2 points against the centralized model (IID) and label skew another 1.8 (Tables III, IV). The attacker lowers FedAvg's macro F1 by 1.11 ± 0.46 points (0.72–1.62), so **H1 is not supported** at full scale; trimmed mean ends 0.36 points *lower* than FedAvg under

TABLE IV
PAIRED DIFFERENCES IN MACRO F1 (POINTS; MEAN ± STD, RANGE)

Comparison	Mean ± std	Range
Non-IID vs. IID (FedAvg)	-1.80 ± 0.17	$[-1.98, -1.63]$
Attack cost, FedAvg	-1.11 ± 0.46	$[-1.62, -0.72]$
Under attack: FedAvg → mean	$+0.28 \pm 0.32$	$[+0.02, +0.63]$
Under attack: mean → trimmed	-0.64 ± 0.23	$[-0.91, -0.50]$

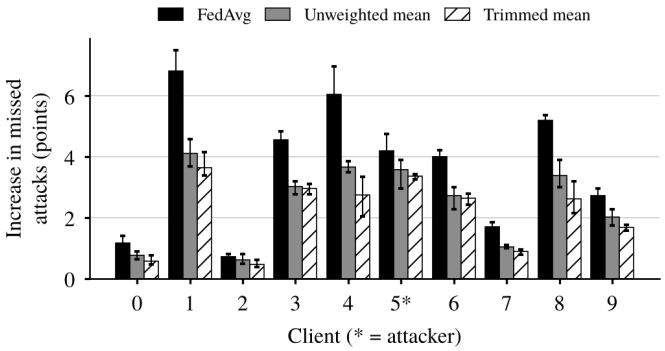


Fig. 2. Per-client increase in missed attacks (binary FNR, attacked minus clean, same rule and seed; bars show the range over 3 seeds).

attack, so **H2 is not supported** either (attack costs of the other rules: Table V). Macro F1 understates the attack: the miss rate rises from 7.2% to 10.6% (+49% relative) while the false-alarm rate falls from 25.3% to 17.4%. The attack moves the decision boundary toward Benign, so more attacks are missed and fewer benign flows are flagged, and the two errors partly cancel in F1.

Weighting vs. trimming. Under attack, dropping size weighting changes macro F1 by $+0.28 \pm 0.32$ points and trimming on top by -0.64 ± 0.23. All three rules end at the same miss rate (10.5–10.6%); the robust rules' smaller *increases* come from higher clean miss rates (8.2–8.4% vs. 7.2%), not from blocking the attacker.

Per family. The attack hits exactly the families that already resemble benign traffic (Fig. 1). Under FedAvg the share predicted Benign rises by 6.6 points for Recon (13.2%→19.8%) and Web (15.7%→22.3%), 7.5 for BruteForce (20.7%→28.2%) and 8.2 for Spoofing (14.3%→22.6%), consistent across seeds (std $\leq$0.9). DDoS, DoS and Mirai are never predicted Benign, attacked or not. Spoofing, the attacker's largest family, rises most, but BruteForce, of which it holds only 154 rows, rises almost as much: the attack shifts the shared benign boundary, not only the attacker's families. The defenses leave Spoofing worse than FedAvg under attack (25.5–25.6% vs. 22.6%).

Per client. Every client's held-out miss rate rises under attack (Fig. 2): by 0.7 to 6.8 points under FedAvg (mean 3.7). The largest increases are at clients 1, 4 and 8, whose attack traffic is 75–86% Recon, Web and BruteForce, not at the attacker. For the worst-off client, label skew matters far more than the attack (macro F1 .692 IID vs. .471 non-IID; attack effect $<$.02).

TABLE V
CHECKS: PAIRED MACRO-F1 CHANGE UNDER ATTACK (POINTS) / MISS RATE UNDER ATTACK (%)

Setting	FedAvg	Unw. mean	Trimmed
Clean miss rate	7.2	8.2	8.4
1 attacker (main)	-1.1 / 10.6	-0.7 / 10.6	-1.1 / 10.5
2 attackers (5, 9)	-2.5 / 14.6	-1.8 / 13.2	-2.0 / 12.6
3 attackers (5, 7, 9)*	-4.1 / 18.7	-2.8 / 15.6	-3.4 / 15.7
60 rounds, 1 attacker	-1.0 / 9.6	–	-1.2 / 9.0
No IAT, 1 attacker†	-1.3 / 16.1	–	–

*Stress test: more attackers than values trimmed per side. Clean miss rates at 60 rounds: 6.7 (FedAvg), 7.2 (trimmed). †Different subset and partition (attacker: client 1; clean miss rate 11.9); compare within the row only.

A. Robustness checks

More attackers (Table V). FedAvg loses 1.1, 2.5 and 4.1 points with 1, 2 and 3 attackers; H1's threshold is crossed only in the 3-attacker stress test (18.7% missed). With 2 and 3 attackers both robust rules miss fewer attacks than FedAvg (trimmed vs. FedAvg, paired: -2.0 ± 0.2 and -3.0 ± 1.1 points), but the gain comes from dropping size weighting: trimming on top of the unweighted mean adds nothing to macro F1 (-0.44 and -0.81 points), even with 2 attackers, inside its trimming budget. *Longer training* (60 rounds) raises clean FedAvg to .715 but leaves the attack cost at -0.96 ± 0.44 points. *Without IAT*, centralized macro F1 is unchanged (.745 vs. .741) and the attack cost is similar (-1.25 ± 0.23 points; miss rate $+4.3$), so IAT is not driving the results. Its subset (305,889 rows) and partition differ slightly, so its federated baseline (.636) is not comparable.

V. DISCUSSION AND NEXT STEPS

With nine honest clients holding 85% of the data, one attacker shifts the benign boundary rather than breaking the model; the pilot's 12-point loss more likely reflects its small subset and 5 rounds. Trimming likely fails because label flipping does not make updates extreme per coordinate; with more attackers, removing size weighting helps because the attackers are the largest clients.

Next (final report): median aggregation; randomly chosen attackers instead of the largest clients, to separate size from poisoning; per-family analysis of the sweep; and one final test-set evaluation. *Limitations:* one testbed with simulated clients; files are not independent captures; capped prevalence; one attack type; three seeds (paired comparisons only).

REFERENCES

[1] B. McMahan *et al.*, "Communication-efficient learning of deep networks from decentralized data," *Proc. AISTATS*, 2017.
[2] V. Tolpegin, S. Truex, M. E. Gursoy, and L. Liu, "Data poisoning attacks against federated learning systems," *Proc. ESORICS*, 2020.
[3] E. C. P. Neto *et al.*, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, 2023.
[4] D. Yin, Y. Chen, K. Ramchandran, and P. Bartlett, "Byzantine-robust distributed learning," *Proc. ICML*, 2018.
[5] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," arXiv:1909.06335, 2019.
[6] D. J. Beutel *et al.*, "Flower: A friendly federated learning framework," arXiv:2007.14390, 2020.